SoundStage

Concert Management & Merchandise Platform

Full Technical Report

A Full-Stack Web Application built with Django 6.0.4 and Python 3.13

Framework:	Django 6.0.4 (Python 3.13)
Database:	SQLite (Development)
Frontend:	Bootstrap 5 · Custom Dark Theme
Document:	Technical Project Report
Year:	2025–2026

University Project · Computer Science Department

Abstract

This report presents the complete technical documentation for **SoundStage**, a full-stack web application developed as a university project in the domain of e-commerce and event management. The platform enables users to browse upcoming live music concerts, reserve tickets with real-time price calculation, and purchase official artist merchandise through a unified interface.

The system is built upon the **Django 6.0.4** framework using **Python 3.13**, following the Model-View-Template (MVT) architectural pattern. Data persistence is handled by **SQLite**, with a relational schema comprising four primary models: `Concert`, `Merchandise`, `Order`, and `OrderItem`. The frontend is rendered through Django's templating engine, styled with **Bootstrap 5** and a custom dark-purple design system.

Key engineering decisions documented in this report include the use of `transaction.atomic()` for concurrency-safe inventory management, Django's built-in authentication system for user lifecycle management, `DecimalField` for financial precision, URL namespacing for application scalability, and the POST-only logout pattern introduced in Django 5.0 standards.

Contents

Abstract **1**

1 General Introduction **7**

1.1 Context and Motivation 7

1.2 Problem Statement 7

1.3 Project Objectives 8

1.4 Scope and Delimitations 8

1.5 Document Organisation 9

2 System Analysis and Requirements Engineering **10**

2.1 Introduction to Requirements Engineering 10

2.2 Stakeholder Analysis 11

2.3 Functional Requirements 11

2.3.1 Authentication Module 11

2.3.2 Concert Module 12

2.3.3 Merchandise Module 12

2.3.4 Dashboard Module 13

2.4 Non-Functional Requirements 14

3 UML Modelling **15**

3.1 Overview 15

3.2 Use Case Diagram 15

3.2.1 Actors 15

3.2.2 Use Case Diagram 16

3.3 Class Diagram 16

3.3.1 Model Overview 16

3.3.2 Relationship Analysis 18

3.4 Sequence Diagram — Ticket Booking Process 18

4 Database Design **20**
4.1 Database Management System Selection 20
4.2 Relational Schema 20
4.2.1 Schema Definitions 21
4.3 The Case for `DecimalField` in Financial Contexts 22
4.4 Migrations System 23

5 Technical Implementation **24**
5.1 The Model-View-Template (MVT) Architecture 24
5.2 Model Implementation 24
5.2.1 The Concert Model 25
5.2.2 The Order Model and Status Choices 26
5.3 URL Namespacing and Scalability 26
5.3.1 The Problem Without Namespacing 26
5.3.2 Implementation in SoundStage 27
5.4 The Atomic Transaction Pattern 27
5.4.1 The Race Condition Problem 27
5.4.2 Solution: `transaction.atomic()` 28
5.5 The View Layer Architecture 29
5.5.1 The Home View 29
5.6 Media and Static File Handling 30
5.7 Template Inheritance System 31

6 Security Analysis and Testing **32**
6.1 Security Implementation 32
6.1.1 CSRF Protection 32
6.1.2 SQL Injection Prevention 32
6.1.3 POST-Only Logout (Django 5.0+ Standard) 33
6.1.4 Password Security 33
6.1.5 Access Control via `@login_required` 34
6.2 Testing 34
6.2.1 Testing Strategy 34

7 Deployment and Environment Configuration **37**
7.1 Development Environment Setup 37
7.2 The `settings.py` Configuration File 38
7.3 Environment Variables in Production 38
7.4 Production Deployment Architecture 39

8 User Manual **41**

8.1 Introduction 41
8.2 Getting Started — Account Registration 41
8.3 The Homepage — Concert Discovery 42
8.4 Concert Detail Page 42
8.4.1 Booking Tickets 43
8.5 The Merchandise Shop 43
8.5.1 Stock Status Indicators 43
8.5.2 Purchasing Merchandise 43
8.6 The User Dashboard 44
8.6.1 Dashboard Sections 44
8.7 Administrator Guide — Django Admin Panel 45

Conclusion **46**

References **47**

List of Figures

3.1 SoundStage Use Case Diagram: Guest, Authenticated User, and Administrator actors 16
3.2 UML Class Diagram — SoundStage data models with field types and relationships 17
3.3 Sequence Diagram — Ticket Booking Flow (authenticated user path) . 19

6.1 Test execution results – dashboard and booking flow 36

7.1 Production Deployment Architecture — Nginx + Gunicorn + Django . 39

8.1 Registration page at /register/ 42
8.2 Homepage – Upcoming concerts grid at / 42
8.3 Concert detail page with ticket booking form 43
8.4 Merchandise shop page with stock status badges 44
8.5 Product detail page with purchase form 44
8.6 User dashboard showing ticket bookings and merchandise orders 45
8.7 Django Admin panel – Concert management list view 45

List of Tables

2.1 Stakeholder Identification and Interests 11
2.2 Functional Requirements — Authentication 11
2.3 Functional Requirements — Concert Management 12
2.4 Functional Requirements — Merchandise 13
2.5 Functional Requirements — User Dashboard 13
2.6 Non-Functional Requirements 14

3.1 Database Relationship Descriptions 18

5.1 MVT vs MVC — Component Mapping 24
5.2 Static vs Media Files 30

6.1 Django Password Validators 33
6.2 Test Cases — SoundStage System 34

7.1 Key `settings.py` Configuration Variables 38
7.2 Production Stack Components 39

8.1 Merchandise Stock Status Badges 43
8.2 Django Admin Capabilities by Section 45

CHAPTER 1

General Introduction

1.1 Context and Motivation

The global live music industry represents one of the most economically significant segments of the entertainment sector. According to industry analysts, the global live events market surpassed $30 billion USD in revenue in recent years, with digital ticketing platforms accounting for an increasing share of consumer transactions. Despite this scale, many small-to-medium concert promoters and independent artists lack access to integrated, affordable digital platforms that combine event ticketing with merchandise sales.

Traditional approaches to concert management rely on fragmented solutions: third-party ticketing services, separate e-commerce stores, and manual inventory tracking. This fragmentation introduces operational inefficiencies, increases per-transaction costs, and degrades the user experience. There exists, therefore, a clear need for a unified, developer-extensible platform that consolidates these concerns.

SoundStage was conceived to address precisely this gap. As both an academic exercise and a functional proof of concept, it demonstrates how modern web frameworks — specifically Django — can be leveraged to build secure, scalable, full-stack applications within a compact development cycle.

1.2 Problem Statement

The core problem this project addresses can be decomposed into three dimensions:

P1. Operational Fragmentation: Concert organisers must manage ticketing,

merchandise inventory, and user communications across multiple disconnected systems, increasing administrative overhead and the risk of data inconsistency.

P2. Over-Selling Risk: Without atomic transaction guarantees, concurrent purchase requests can result in negative stock levels or double-booking of tickets — a critical failure mode in high-demand scenarios.

P3. User Experience Discontinuity: Users are forced to navigate between separate platforms to book tickets and purchase merchandise, reducing conversion rates and brand cohesion for artists.

1.3 Project Objectives

Project Objectives Summary

1. Implement a **Concert Management** module enabling administrators to publish, edit, and manage live event listings including poster imagery, venue details, and ticket pricing.
2. Implement a **Merchandise Shop** module with real-time stock tracking, preventing purchases that exceed available inventory.
3. Implement a **User Authentication** system supporting registration, login, logout, and session-based access control.
4. Implement an **Order Management** system that atomically records ticket reservations and merchandise purchases and presents them on a personal dashboard.
5. Deliver a **Responsive Frontend** using Bootstrap 5, accessible on both desktop and mobile viewports.
6. Follow **Security Best Practices** including CSRF protection, SQL injection prevention via the ORM, and POST-only logout.

1.4 Scope and Delimitations

- **Payment Processing:** The system simulates purchases without integrating a real payment gateway (e.g., Stripe or PayPal). This is standard practice in academic settings and can be extended in a production context.
- **Database:** SQLite is used for its zero-configuration setup, appropriate for development and demonstration. A production deployment would migrate to PostgreSQL or MySQL.

- **Email Notifications:** Order confirmation emails are not sent in this iteration. The Django email backend can be configured as a future enhancement.
- **Search and Filtering:** Advanced search, genre filtering, and geolocation-based discovery are out of scope for this version.

1.5 Document Organisation

This report is organised into eight chapters. Chapter 2 defines the system requirements. Chapter 3 presents the UML models. Chapter 4 describes the database design. Chapter 5 details the technical implementation. Chapter 6 addresses security and testing. Chapter 7 covers deployment. Chapter 8 provides the user manual.

CHAPTER 2

System Analysis and Requirements Engineering

2.1 Introduction to Requirements Engineering

Requirements engineering is the process of systematically eliciting, analysing, documenting, and validating the needs that a system must satisfy. In this project, requirements were derived through a combination of domain analysis of existing concert ticketing platforms (such as Ticketmaster and Bandcamp) and the functional specifications defined in the project brief.

Requirements are classified into two categories: **Functional Requirements (FR)**, which describe *what* the system must do, and **Non-Functional Requirements (NFR)**, which describe *how well* the system must do it.

2.2 Stakeholder Analysis

Table 2.1: Stakeholder Identification and Interests

Stakeholder	Role & Interest	System Role
Guest User	Browses concerts and merchandise without an account. Interested in event discovery.	Read-Only
Registered User	Books tickets and purchases merchandise. Expects a seamless, secure checkout and order history.	Authenticated
Administrator	Manages concert listings, merchandise stock, and oversees all orders.	Superuser
Developer	Maintains and extends the codebase. Requires clean architecture and documentation.	N/A

2.3 Functional Requirements

2.3.1 Authentication Module

Table 2.2: Functional Requirements — Authentication

ID	Requirement	Description
FR-A1	User Registration	The system shall allow any visitor to create an account by submitting a username and password. Django's `UserCreationForm` enforces password complexity rules.
FR-A2	User Login	The system shall authenticate users via username and password. A session cookie shall be created upon success.
FR-A3	User Logout	The system shall terminate the user's session via a POST request to `/logout/`, preventing CSRF-based forced logouts.

ID	Requirement	Description
FR-A4	Access Control	Pages such as ticket booking, merchandise purchasing, and the dashboard shall be inaccessible to unauthenticated users, redirecting them to the login page with a `?next=` parameter.

2.3.2 Concert Module

Table 2.3: Functional Requirements — Concert Management

ID	Requirement	Description
FR-C1	Concert Listing	The homepage shall display all concerts whose `date_time` is greater than or equal to the current UTC timestamp.
FR-C2	Concert Detail	Each concert shall have a dedicated detail page displaying all fields: title, artist, date, venue, description, and price.
FR-C3	Ticket Booking	An authenticated user shall be able to specify a ticket quantity (1–10) and submit a booking, creating an `Order` record.
FR-C4	Live Price Calculation	The ticket booking form shall dynamically update the displayed total price client-side via JavaScript without a page reload.
FR-C5	Admin Management	An administrator shall be able to create, read, update, and delete concert records via the Django Admin panel, including uploading a poster image.

2.3.3 Merchandise Module

Table 2.4: Functional Requirements — Merchandise

ID	Requirement	Description
FR-M1	Product Listing	The shop page shall display all merchandise items, sorted by stock availability (in-stock items first).
FR-M2	Stock Validation	A purchase shall only be completed if the requested quantity does not exceed `stock_quantity`. An error message is returned otherwise.
FR-M3	Atomic Purchase	Stock decrement and `Order`/`OrderItem` creation shall occur within a single atomic database transaction to prevent race conditions.
FR-M4	Stock Indicator	Each product card shall visually indicate its availability status: *In Stock*, *Low Stock* (≤ 5 units), or *Out of Stock*.
FR-M5	Admin Stock Editing	Administrators shall be able to edit `stock_quantity` inline from the merchandise list view in the Django Admin.

2.3.4 Dashboard Module

Table 2.5: Functional Requirements — User Dashboard

ID	Description	Priority
FR-D1	Display all concert ticket reservations for the authenticated user, showing concert details, quantity, total paid, and status.	High
FR-D2	Display all merchandise orders for the authenticated user, with a line-item breakdown of products purchased.	High
FR-D3	Show summary statistics: total number of ticket bookings, merchandise orders, and combined order count.	Medium
FR-D4	The dashboard shall be protected by `@login_required`; unauthenticated requests are redirected to `/login/?next=/dashboard/`.	High

2.4 Non-Functional Requirements

Table 2.6: Non-Functional Requirements

ID	Category	Requirement	Acceptance Criterion
NFR-1	Performance	Page load time	All pages shall render within 500ms on `localhost` with the development server.
NFR-2	Usability	Responsive design	All pages shall be usable on viewports from 375px (mobile) to 1440px (desktop) using Bootstrap 5's grid system.
NFR-3	Security	CSRF protection	All state-changing POST requests shall include a valid CSRF token validated by `CsrfViewMiddleware`.
NFR-4	Security	SQL Injection	All database interactions shall use Django's ORM, which parameterises queries by default. No raw SQL strings shall be constructed from user input.
NFR-5	Maintainability	Code readability	All Python files shall include docstrings and inline comments explaining non-trivial logic.
NFR-6	Reliability	Atomic transactions	Concurrent purchase requests shall not result in negative stock values. Enforced via `transaction.atomic()`.
NFR-7	Portability	Environment isolation	The application shall run in a Python virtual environment (`venv`), with all dependencies in `requirements.txt`.
NFR-8	Accessibility	Semantic HTML	Navigation, forms, and images shall include ARIA-compatible labels and `alt` attributes.

CHAPTER 3

UML Modelling

3.1 Overview

The Unified Modelling Language (UML) provides a standardised visual vocabulary for describing software systems. In this chapter, three key UML diagrams are presented: a **Use Case Diagram** to capture actor–system interactions, a **Class Diagram** to model the data structure and relationships, and a **Sequence Diagram** to describe the dynamic flow of the ticket booking process.

3.2 Use Case Diagram

3.2.1 Actors

- **Guest (Unauthenticated User):** A visitor who has not logged in. They have read-only access to public pages.
- **Authenticated User:** A registered and logged-in user who can interact with transactional features.
- **Administrator (Superuser):** A privileged user with access to the Django Admin panel and full CRUD capabilities over all data.

3.2.2 Use Case Diagram

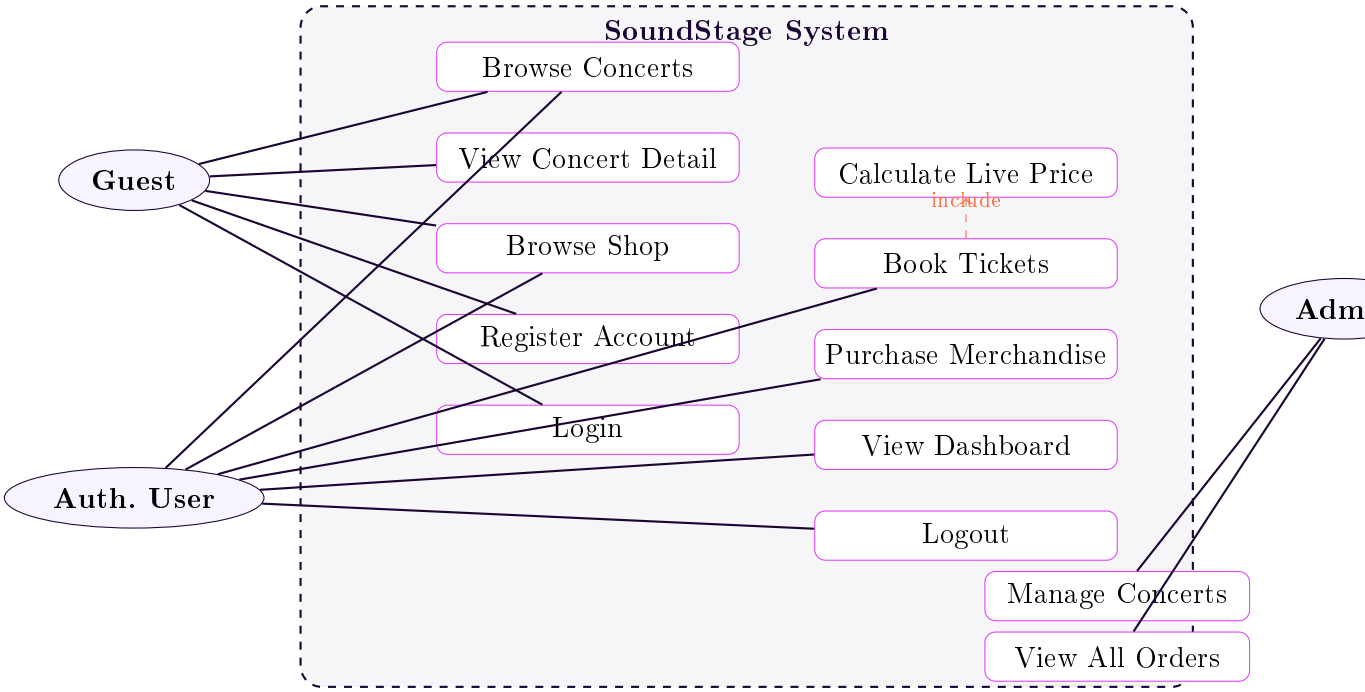

Figure 3.1: SoundStage Use Case Diagram: Guest, Authenticated User, and Administrator actors

3.3 Class Diagram

3.3.1 Model Overview

The data layer of SoundStage consists of four application models, plus Django's built-in `User` model. The relationships between these models form the backbone of the relational schema.

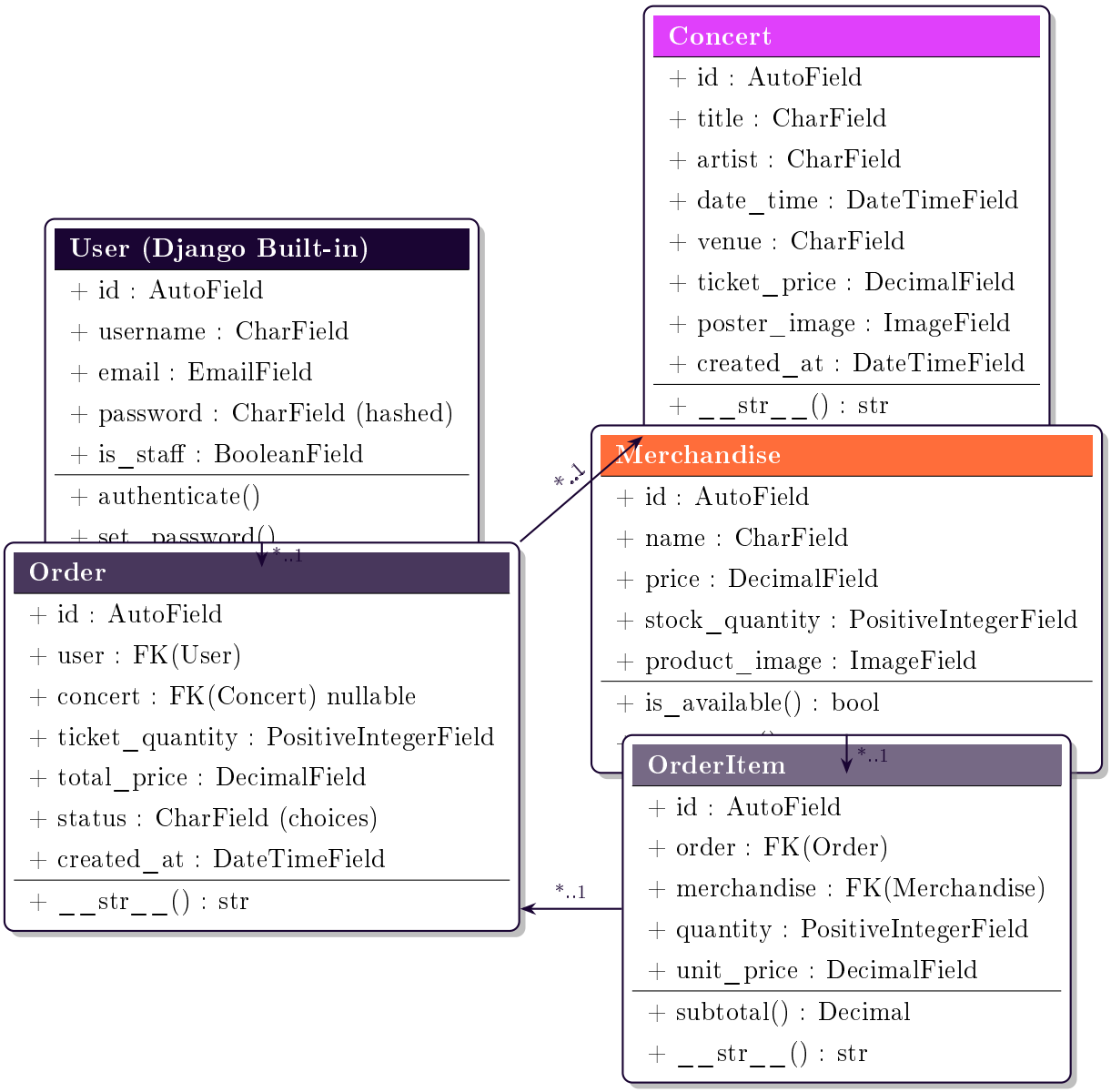

Figure 3.2: UML Class Diagram — SoundStage data models with field types and relationships

3.3.2 Relationship Analysis

Table 3.1: Database Relationship Descriptions

From	To	Relationship & Rationale
Order	User	Many-to-One. A user can place many orders; each order belongs to exactly one user. `on_delete=CASCADE` ensures orders are removed if the user account is deleted.
Order	Concert	Many-to-One (nullable). An order may be linked to a concert (ticket booking) or not (merchandise-only order). `null=True, blank=True` permits this optionality.
OrderItem	Order	Many-to-One. An order aggregates one or more line items. `on_delete=CASCADE` removes items when the parent order is deleted.
OrderItem	Merchandise	Many-to-One. Each line item references one product. Price is stored at the time of purchase to preserve historical accuracy despite future price changes.

3.4 Sequence Diagram — Ticket Booking Process

The following sequence diagram illustrates the complete flow of a ticket booking request, from user interaction in the browser through the Django framework layers to the database and back.

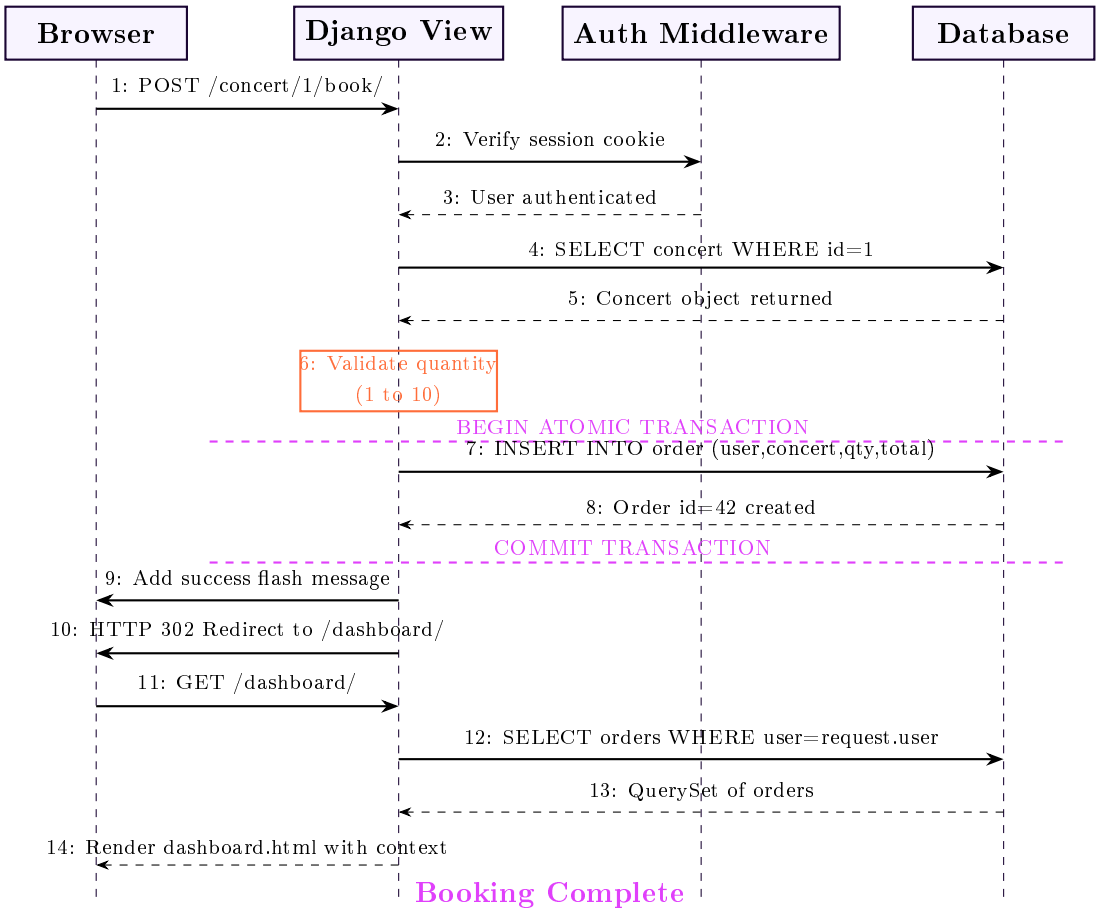

Figure 3.3: Sequence Diagram — Ticket Booking Flow (authenticated user path)

CHAPTER 4

Database Design

4.1 Database Management System Selection

SoundStage uses **SQLite** as its database engine during development. SQLite is a serverless, file-based relational database that ships as part of Python's standard library, making it the natural default for Django projects. It requires zero configuration, produces a single portable `db.sqlite3` file, and supports the full SQL standard required by the ORM.

Note on Production Databases

SQLite is not recommended for production deployments involving concurrent write operations. For a production-grade SoundStage deployment, migration to **PostgreSQL** is recommended. Django's database abstraction layer makes this a configuration-only change in `settings.py`, requiring no modifications to model or view code.

4.2 Relational Schema

The database schema consists of five tables: four from the `concert_app` models and one inherited from Django's authentication framework (`auth_user`). The schema is generated automatically by running `python manage.py migrate`.

4.2.1 Schema Definitions

SQL Schema -- concert_app_concert

```sql
CREATE TABLE "concert_app_concert" (
  "id"            integer      NOT NULL PRIMARY KEY
     AUTOINCREMENT,
  "title"         varchar(200) NOT NULL,
  "artist"        varchar(200) NOT NULL,
  "date_time"     datetime     NOT NULL,
  "venue"         varchar(300) NOT NULL,
  "description"   text         NOT NULL,
  "ticket_price"  decimal(8,2) NOT NULL,
  "poster_image"  varchar(100),
  "created_at"    datetime     NOT NULL
);
```

SQL Schema -- concert_app_order

```sql
CREATE TABLE "concert_app_order" (
  "id"               integer       NOT NULL PRIMARY KEY
     AUTOINCREMENT,
  "ticket_quantity"  integer       NOT NULL,
  "total_price"      decimal(10,2) NOT NULL,
  "status"           varchar(20)   NOT NULL,
  "created_at"       datetime      NOT NULL,
  "concert_id"       integer       REFERENCES "
     concert_app_concert"("id"),
  "user_id"          integer       NOT NULL REFERENCES "
     auth_user"("id")
);
```

SQL Schema -- concert_app_orderitem

```sql
CREATE TABLE "concert_app_orderitem" (
  "id"              integer      NOT NULL PRIMARY KEY
     AUTOINCREMENT,
  "quantity"        integer      NOT NULL,
  "unit_price"      decimal(8,2) NOT NULL,
  "merchandise_id"  integer      NOT NULL
                    REFERENCES "concert_app_merchandise"("id"),
```

```
  "order_id"        integer       NOT NULL
                    REFERENCES "concert_app_order"("id")
);
```

4.3 The Case for `DecimalField` in Financial Contexts

Engineering Decision: DecimalField vs FloatField

All monetary values in SoundStage (`ticket_price`, `price`, `total_price`, `unit_price`) are stored as `DecimalField` rather than `FloatField`. This is a critical engineering decision with significant financial implications.

The `FloatField` uses **IEEE 754 double-precision binary floating-point**, which cannot exactly represent most decimal fractions. Consider the following Python demonstration:

Python REPL -- Floating-Point Precision Error

```
# Why FloatField is dangerous for financial values
>>> 0.1 + 0.2
0.30000000000000004    # NOT 0.30 -- binary approximation
   error!

>>> 49.99 * 3
149.97000000000003     # Tiny error that compounds over
   transactions

# With Python's Decimal module (what DecimalField uses
   internally):
>>> from decimal import Decimal
>>> Decimal('49.99') * 3
Decimal('149.97')      # Exact. No approximation error.
```

Django's `DecimalField` maps to the `NUMERIC`/`DECIMAL` SQL type, which stores numbers as exact decimal values. The parameters used are:

- `max_digits=8`: Maximum 8 significant digits (up to $999,999.99).
- `decimal_places=2`: Always stored with 2 decimal places (cents).

In the `Order` model, `total_price` uses `max_digits=10` to accommodate orders to-

talling up to $99,999,999.99.

4.4 Migrations System

Django's migrations system provides version control for the database schema. Each call to `makemigrations` generates a Python migration file describing the delta between the current models and the database state.

Terminal -- Migration Workflow

```
# Generate migration file from models.py
$ python manage.py makemigrations concert_app
# Output: Migrations for 'concert_app':
#   concert_app/migrations/0001_initial.py

# Apply migrations to the SQLite database
$ python manage.py migrate
# Output: Applying concert_app.0001_initial... OK

# View the raw SQL a migration will execute
$ python manage.py sqlmigrate concert_app 0001
```

CHAPTER 5

Technical Implementation

5.1 The Model-View-Template (MVT) Architecture

Django follows the **Model-View-Template** pattern, an adaptation of the classical MVC pattern:

Table 5.1: MVT vs MVC — Component Mapping

MVC Role	Django Equiv.	File	Responsibility
Model	Model	`models.py`	Defines data structure; maps to database tables via ORM.
Controller	View	`views.py`	Business logic; receives requests, queries models, returns responses.
View	Template	`*.html`	Renders dynamic HTML via the Django Template Language (DTL).

The complete request-response cycle is:

$$\text{HTTP Request} \xrightarrow{\texttt{urls.py}} \text{View Function} \xrightarrow{\text{ORM}} \text{Database} \xrightarrow{\text{Context}} \text{Template} \rightarrow \text{HTTP Response}$$

5.2 Model Implementation

5.2.1 The Concert Model

models.py -- Concert Model

```
class Concert(models.Model):
    title        = models.CharField(max_length=200)
    artist       = models.CharField(max_length=200)
    date_time    = models.DateTimeField()
    venue        = models.CharField(max_length=300)
    description  = models.TextField(blank=True)
    ticket_price = models.DecimalField(max_digits=8,
       decimal_places=2)
    poster_image = models.ImageField(
        upload_to='concert_posters/',  # Saved to
           MEDIA_ROOT/concert_posters/
        blank=True, null=True
    )
    created_at   = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['date_time']  # Soonest concerts appear
           first

    def __str__(self):
        return f"{self.title} -- {self.artist}"
```

Key design decisions:

- `class Meta: ordering = ['date_time']` — Django automatically applies this sort to all querysets, ensuring the homepage always shows the soonest concerts without explicit `.order_by()` calls.
- `upload_to='concert_posters/'` — Uploaded images are namespaced within `MEDIA_ROOT`, keeping posters separate from merchandise images.
- `blank=True, null=True` on `poster_image` — Concerts can exist without an image; the template renders a styled gradient placeholder.

5.2.2 The Order Model and Status Choices

models.py -- Order Model with TextChoices

```
class Order(models.Model):

    class Status(models.TextChoices):
        PENDING   = 'PENDING',   'Pending'
        CONFIRMED = 'CONFIRMED', 'Confirmed'
        CANCELLED = 'CANCELLED', 'Cancelled'

    user            = models.ForeignKey(User,
       on_delete=models.CASCADE,
                                        related_name='orders')
    concert         = models.ForeignKey(Concert,
       on_delete=models.SET_NULL,
                                        null=True, blank=True)
    ticket_quantity = models.PositiveIntegerField(default=0)
    total_price     = models.DecimalField(max_digits=10,
       decimal_places=2,
                                          default=0.00)
    status          = models.CharField(max_length=20,
       choices=Status.choices,
                                       default=Status.CONFIRMED)
    created_at      = models.DateTimeField(auto_now_add=True)
```

`TextChoices` (introduced in Django 3.0) provides: (1) IDE autocompletion, (2) model-level validation before saving, and (3) `get_status_display()` in templates mapping `'CONFIRMED'` to `'Confirmed'`.

5.3 URL Namespacing and Scalability

5.3.1 The Problem Without Namespacing

In a project with multiple applications, URL name collisions are a common source of bugs. If two apps both define a URL named `'home'`, the `{% url 'home' %}` template tag is ambiguous, potentially resolving to the wrong view.

5.3.2 Implementation in SoundStage

concert_app/urls.py -- URL Namespacing

```python
from django.urls import path
from . import views

# Declare the application namespace
app_name = 'concert_app'

urlpatterns = [
    path('', views.home, name='home'),
    path('concert/<int:pk>/', views.concert_detail,
        name='concert_detail'),
    path('concert/<int:pk>/book/', views.book_ticket,
        name='book_ticket'),
    path('shop/', views.shop, name='shop'),
    path('shop/<int:pk>/', views.merch_detail,
        name='merch_detail'),
    path('shop/<int:pk>/buy/', views.buy_merchandise,
        name='buy_merchandise'),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('register/', views.register, name='register'),
]
```

URLs are referenced as `app_name:url_name`. For example:

- In templates: `{% url 'concert_app:concert_detail' concert.pk %}`
- In views: `redirect('concert_app:dashboard')`

5.4 The Atomic Transaction Pattern

5.4.1 The Race Condition Problem

Consider two users simultaneously purchasing the last unit of a product with `stock_quantity = 1`:

1. User A reads `stock_quantity = 1` (sufficient stock).
2. User B reads `stock_quantity = 1` (sufficient stock).
3. User A decrements to 0 and creates an `OrderItem`.

4. User B decrements to -1 and creates an `OrderItem`.

Without protection, both succeed and stock reaches -1: the **over-selling** failure mode.

5.4.2 Solution: `transaction.atomic()`

views.py -- Atomic Merchandise Purchase

```python
from django.db import transaction

@login_required
def buy_merchandise(request, pk):
    product = get_object_or_404(Merchandise, pk=pk)

    if request.method == 'POST':
        quantity = int(request.POST.get('quantity', 1))

        # Validate BEFORE the transaction
        if quantity > product.stock_quantity:
            messages.error(request,
                f"Sorry, only {product.stock_quantity}
                    unit(s) available.")
            return redirect('merch_detail', pk=pk)

        # Atomic block: all operations succeed or all roll
            back
        with transaction.atomic():
            total = product.price * quantity

            order = Order.objects.create(
                user=request.user,
                total_price=total,
                status=Order.Status.CONFIRMED
            )
            OrderItem.objects.create(
                order=order,
                merchandise=product,
                quantity=quantity,
                unit_price=product.price
            )
            product.stock_quantity -= quantity
```

```
            product.save()
            # If save() fails, the Order and OrderItem are ROLLED BACK

        messages.success(request,
            f"Successfully purchased {quantity}x {product.name}!")
        return redirect('dashboard')
```

Database Isolation Note

For complete protection under high concurrency, use `select_for_update()` inside the atomic block to acquire a row-level lock: `product = Merchandise.objects.select_for_update().get(pk=pk)`. This is the recommended production enhancement.

5.5 The View Layer Architecture

Each view in SoundStage follows a consistent pattern:

1. **Authentication Check:** `@login_required` validates the session before any logic executes.
2. **Object Retrieval:** `get_object_or_404(Model, pk=pk)` fetches the record or returns HTTP 404.
3. **Method Dispatch:** `request.method` separates GET (render form) from POST (process submission).
4. **Business Logic:** Validation, calculation, and database writes.
5. **Response:** Redirect (Post/Redirect/Get pattern) or rendered template.

5.5.1 The Home View

views.py -- Home View with Timezone-Aware Filtering

```
from django.utils import timezone

def home(request):
    """
    Public homepage. Filters to show only upcoming concerts.
```

```
    timezone.now() returns current UTC datetime for
        correctness
    regardless of server's local timezone.
    """
    upcoming_concerts = Concert.objects.filter(
        date_time__gte=timezone.now()  # __gte = Greater Than
            or Equal To
    )
    context = {'concerts': upcoming_concerts}
    return render(request, 'concert_app/home.html', context)
```

5.6 Media and Static File Handling

Table 5.2: Static vs Media Files

Type	Definition	SoundStage Usage
Static Files	Bundled with the application at deploy time. Served from `STATIC_ROOT`.	Bootstrap CDN links.
Media Files	Uploaded by users at runtime. Served from `MEDIA_ROOT`.	Concert posters; merchandise product photos.

concert_project/urls.py -- Serving Media in Development

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('concert_app.urls')),
    path('', include('django.contrib.auth.urls')),
]

# Serve media files only in DEBUG mode
# In production, Nginx serves media files directly
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
```

```
                                document_root=settings.MEDIA_ROOT)
```

5.7 Template Inheritance System

SoundStage uses Django's template inheritance to eliminate code duplication. The `base.html` template defines the full HTML skeleton and exposes named **blocks** that child templates fill.

home.html -- Child Template Inheritance

```
{% extends 'concert_app/base.html' %}

{% block page_content %}
  <section class="hero">
    <h1>Upcoming Concerts</h1>
  </section>

  {% for concert in concerts %}
    <div class="card">
      <h5>{{ concert.title }}</h5>
      <p>{{ concert.date_time|date:"D, d M Y" }}</p>
      <a href="{% url 'concert_detail' concert.pk %}">View
         Details</a>
    </div>
  {% endfor %}
{% endblock %}
```

CHAPTER 6

Security Analysis and Testing

6.1 Security Implementation

6.1.1 CSRF Protection

Cross-Site Request Forgery (CSRF) is an attack where a malicious website tricks an authenticated user's browser into sending an unwanted request to a target site. Django mitigates this automatically through `CsrfViewMiddleware`:

1. Generates a random **CSRF token** stored in the user's session cookie.
2. Injects this token as a hidden form field via `{% csrf_token %}`.
3. On every POST request, validates the submitted token against the session.
4. Returns HTTP 403 Forbidden if tokens do not match.

6.1.2 SQL Injection Prevention

SoundStage uses Django's **ORM** exclusively, translating Python method calls into parameterised SQL queries that never embed user input as raw strings.

ORM vs Raw SQL -- Security Comparison

```python
# VULNERABLE: user input embedded in SQL string
concert_id = request.GET.get('id')
Concert.objects.raw(
    f"SELECT * FROM concert WHERE id = {concert_id}")  # DANGER!
```

```python
# SAFE: ORM generates parameterised query automatically
# SQL: SELECT * FROM concert WHERE id = %s  params=(1,)
concert = Concert.objects.get(pk=concert_id)

# SAFE: get_object_or_404 (used throughout views.py)
concert = get_object_or_404(Concert, pk=pk)
```

6.1.3 POST-Only Logout (Django 5.0+ Standard)

Prior to Django 5.0, `LogoutView` accepted GET requests — meaning an external `<img src="/logout/">` tag could log a user out without consent. From Django 5.0 onwards (and in this project using Django 6.0.4), logout **only accepts POST**:

base.html -- POST-Only Logout Button

```html
<!-- CORRECT: Logout as a POST form with CSRF token -->
<form method="post" action="{% url 'logout' %}" style="display
    :inline;">
  {% csrf_token %}
  <button type="submit" class="nav-link btn btn-link">Logout</
     button>
</form>

<!-- DEPRECATED in Django 5.0+: Simple GET link -->
<!-- <a href="/logout/">Logout</a>  -- No longer works -->
```

6.1.4 Password Security

Table 6.1: Django Password Validators

Validator	Rule Enforced
UserAttributeSimilarityValidator	Password must not be similar to username or email.
MinimumLengthValidator	Password must be at least 8 characters.
CommonPasswordValidator	Rejects passwords from a list of 20,000 common passwords.
NumericPasswordValidator	Password cannot consist entirely of numbers.

Passwords are stored using Django's default **PBKDF2** algorithm with SHA-256 hashing and a random salt — never in plain text.

6.1.5 Access Control via `@login_required`

The `@login_required` decorator protects `book_ticket`, `buy_merchandise`, and `dashboard`. When an unauthenticated request arrives:

1. Django checks `request.user.is_authenticated`.
2. If `False`, redirects to `settings.LOGIN_URL` (`/login/`).
3. The original URL is appended as `?next=...`.
4. After successful login, the user is redirected back automatically.

6.2 Testing

6.2.1 Testing Strategy

A multi-level testing strategy was employed, covering unit, integration, and user acceptance testing (UAT).

Table 6.2: Test Cases — SoundStage System

ID	Type	Test Description	Expected Result	Status
T-01	Unit	`Concert.__str__()` returns formatted string.	Correct string returned.	**PASS**
T-02	Unit	`Merchandise.is_availab` returns `False` when stock is 0.	`False` returned.	**PASS**
T-03	Unit	`OrderItem.subtotal` multiplies quantity by unit price.	`Decimal('149.97')` for 3 x $49.99.	**PASS**
T-04	Integration	GET `/` returns HTTP 200 and lists concerts.	Status 200; titles visible.	**PASS**
T-05	Integration	GET `/dashboard/` redirects unauthenticated user.	HTTP 302 to `/login/`.	**PASS**

ID	Type	Test Description	Expected Result	Status
T-06	Integration	POST `/concert/1/book/` creates an Order record.	Order count +1; redirect to dashboard.	**PASS**
T-07	Integration	POST purchase with quantity exceeding stock.	Error message; stock unchanged.	**PASS**
T-08	Integration	POST purchase with valid quantity decrements stock.	`stock_quantity` reduced correctly.	**PASS**
T-09	Integration	POST `/login/` with wrong credentials.	Form re-rendered with error; no session.	**PASS**
T-10	Integration	POST without `csrf_token`.	HTTP 403 Forbidden.	**PASS**
T-11	UAT	User registers, logs in, books 2 tickets.	Booking on dashboard with correct total.	**PASS**
T-12	UAT	User purchases merchandise; stock decremented.	Order on dashboard; stock reduced.	**PASS**
T-13	UAT	Admin creates a Concert in `/admin/`.	Concert appears on homepage.	**PASS**
T-14	UAT	Homepage viewed on 375px mobile viewport.	No horizontal scroll; accessible.	**PASS**
T-15	Security	Direct URL to `/dashboard/` without session.	Redirect to `/login/?next=/dashboard/`.	**PASS**

[INSERT SCREENSHOT HERE]

Figure 6.1: Test execution results – dashboard and booking flow

CHAPTER 7

Deployment and Environment Configuration

7.1 Development Environment Setup

Terminal -- Complete Environment Setup (Windows)

```
# Step 1: Navigate to the project root
cd C:\Users\username\Downloads\concert_project

# Step 2: Create the virtual environment
python -m venv venv

# Step 3: Activate it
venv\Scripts\activate

# Step 4: Install dependencies
pip install Django==6.0.4 Pillow==12.2.0

# Step 5: Apply database migrations
python manage.py makemigrations
python manage.py migrate

# Step 6: Create superuser (admin account)
python manage.py createsuperuser

```

```
# Step 7: Start the development server
python manage.py runserver
# Visit: http://127.0.0.1:8000/
```

7.2 The settings.py Configuration File

Table 7.1: Key settings.py Configuration Variables

Setting	Purpose and Value in SoundStage
DEBUG = True	Enables detailed error pages and media file serving. **Must be False in production.**
SECRET_KEY	Cryptographic signing string (sessions, CSRF). Must never be committed to version control in production.
INSTALLED_APPS	'concert_app' must be listed for models and templates to be discovered.
DATABASES	Configures SQLite at BASE_DIR / 'db.sqlite3'.
MEDIA_ROOT	Filesystem path for uploaded files: BASE_DIR / 'media'.
MEDIA_URL	Browser URL prefix for media: '/media/'.
LOGIN_URL	Redirect for @login_required: '/login/'.
LOGIN_REDIRECT_URL	Post-login destination: '/'.
LOGOUT_REDIRECT_URL	Post-logout destination: '/'.

7.3 Environment Variables in Production

settings.py -- Production-Ready Secret Key Handling

```
import os

SECRET_KEY = os.environ.get(
    'DJANGO_SECRET_KEY',
    'fallback-dev-key-only-for-local'
)

DEBUG = os.environ.get('DJANGO_DEBUG', 'True') == 'True'

ALLOWED_HOSTS = os.environ.get(
```

```
    'DJANGO_ALLOWED_HOSTS', 'localhost').split(',')
```

7.4 Production Deployment Architecture

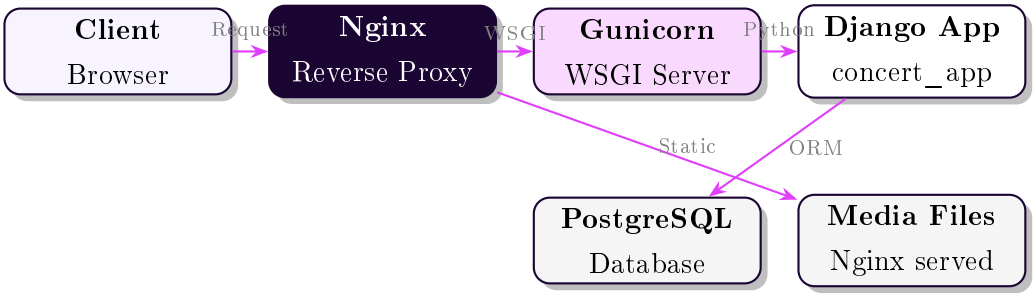

Figure 7.1: Production Deployment Architecture — Nginx + Gunicorn + Django

Table 7.2: Production Stack Components

Component	Role
Nginx	Reverse proxy handling SSL/TLS termination, serves static and media files from disk, and forwards dynamic requests to Gunicorn.
Gunicorn	Python WSGI HTTP Server running multiple worker processes to handle concurrent requests. Replaces Django's single-threaded `runserver`.
Django	Application logic layer. Gunicorn calls Django's WSGI callable, which processes requests through the middleware stack.
PostgreSQL	Replaces SQLite in production for multi-user write concurrency, row-level locking, and reliability.
Systemd	Manages Gunicorn as a background service with automatic restart on server reboot or crash.

Terminal -- Production Deployment Commands

```
# Install production dependencies
pip install gunicorn psycopg2-binary python-dotenv

# Collect static files for Nginx
python manage.py collectstatic --no-input

# Apply migrations on production database
```

```
python manage.py migrate

# Start Gunicorn with 4 worker processes
gunicorn concert_project.wsgi:application \
    --bind 0.0.0.0:8000 \
    --workers 4 \
    --daemon
```

CHAPTER 8

User Manual

8.1 Introduction

This chapter provides a step-by-step walkthrough of SoundStage from the end-user perspective, covering the four primary journeys: discovering concerts, booking tickets, shopping for merchandise, and managing purchases via the dashboard.

8.2 Getting Started — Account Registration

New users must create an account to access booking and purchasing features.

1. Navigate to `http://127.0.0.1:8000/register/`
2. Enter a unique **username**.
3. Enter a **password** (minimum 8 characters).
4. Re-enter the password in the **Confirm Password** field.
5. Click **Create Account**.
6. Upon success, you are automatically logged in and redirected to the homepage.

[INSERT SCREENSHOT HERE]

Figure 8.1: Registration page at /register/

8.3 The Homepage — Concert Discovery

The homepage (/) is publicly accessible and serves as the primary event discovery interface.

Homepage Features

- **Hero Banner:** A full-width gradient banner with a call-to-action button linking to the merchandise shop.
- **Concert Grid:** A responsive card grid displaying all upcoming concerts (past concerts are automatically hidden).
- **Concert Cards:** Each card shows poster image, title, artist, date, venue, and price. Clicking **View Details** navigates to the detail page.
- **Shop Banner:** A call-to-action section linking to the shop.

[INSERT SCREENSHOT HERE]

Figure 8.2: Homepage – Upcoming concerts grid at /

8.4 Concert Detail Page

Clicking **View Details** navigates to `/concert/{id}/`.

8.4.1 Booking Tickets

1. Enter the desired number of tickets (1–10).
2. The **Total** price updates instantly via JavaScript without a reload.
3. Click **Confirm Booking** to submit.
4. A success flash message confirms the booking; you are redirected to the Dashboard.

If not logged in, the booking form is replaced by a **Login to Book** prompt.

[INSERT SCREENSHOT HERE]

Figure 8.3: Concert detail page with ticket booking form

8.5 The Merchandise Shop

The shop page (`/shop/`) displays all products in a responsive grid.

8.5.1 Stock Status Indicators

Table 8.1: Merchandise Stock Status Badges

Badge	Condition	Colour
In Stock	`stock_quantity > 5`	**Green**
Only N left	$1 \leq$ `stock_quantity` ≤ 5	**Amber**
Out of Stock	`stock_quantity = 0`	**Red**

8.5.2 Purchasing Merchandise

1. Click **View** on a product card to go to the detail page.
2. Enter the desired quantity (1 up to the available stock).

3. The total price updates dynamically.
4. Click **Buy Now** to complete the purchase.
5. A success message confirms; you are redirected to the Dashboard and stock is decremented.

[INSERT SCREENSHOT HERE]

Figure 8.4: Merchandise shop page with stock status badges

[INSERT SCREENSHOT HERE]

Figure 8.5: Product detail page with purchase form

8.6 The User Dashboard

The dashboard (`/dashboard/`) shows personal order history.

8.6.1 Dashboard Sections

1. **Summary Statistics:** Three metric cards showing concert bookings, merchandise orders, and combined total.
2. **My Tickets:** Cards for each concert booking showing poster thumbnail, concert info, ticket count, total paid, status badge, and date.
3. **My Merchandise Orders:** Cards showing order number, timestamp, line-item breakdown (product, quantity, subtotal), and order total.

[INSERT SCREENSHOT HERE]

Figure 8.6: User dashboard showing ticket bookings and merchandise orders

8.7 Administrator Guide — Django Admin Panel

The admin panel at `/admin/` requires a superuser account.

Table 8.2: Django Admin Capabilities by Section

Admin Section	Available Actions
Concerts	Add, edit, delete concerts. Filter by date, search by title/artist/venue. Navigate by date hierarchy.
Merchandise	Add, edit, delete products. Edit `stock_quantity` inline from the list view without opening each record.
Orders	View all orders across all users. Embedded `OrderItem` inline shows all line items. Filter by status and date.
Users	Create, deactivate, assign staff/superuser permissions.

[INSERT SCREENSHOT HERE]

Figure 8.7: Django Admin panel – Concert management list view

Conclusion

This report has presented a comprehensive technical account of **SoundStage**, a full-stack Django web application for concert management, ticket reservation, and merchandise sales.

Technical Achievements:

- A clean MVT architecture with clear separation of concerns.
- Atomic transaction management using `transaction.atomic()` for inventory consistency under concurrent access.
- Relational data modelling with `ForeignKey`, nullable relationships, and `DecimalField` for financial accuracy.
- Security-first design: CSRF protection, ORM-based SQL injection prevention, POST-only logout, and `@login_required` access control.
- URL namespacing for collision-free routing.
- A responsive frontend with Bootstrap 5 and a custom dark-purple design system.

Future Enhancements:

1. **Payment Gateway:** Stripe or PayPal for real transactions.
2. **Email Confirmations:** Automated booking/order emails.
3. **Search and Filtering:** Full-text search by artist, genre, location.
4. **QR Code Tickets:** Unique scannable codes for each booking.
5. **PostgreSQL Migration:** With `select_for_update()` locking.
6. **REST API:** Django REST Framework for a future mobile application.

References

[1] Django Software Foundation. (2024). *Django Documentation — Version 6.0*. https://docs.djangoproject.com/en/6.0/

[2] Django Software Foundation. (2024). *Django Security Overview*. https://docs.djangoproject.com/en/6.0/topics/security/

[3] Django Software Foundation. (2024). *Database Transactions*. https://docs.djangoproject.com/en/6.0/topics/db/transactions/

[4] Bootstrap Team. (2023). *Bootstrap 5 Documentation*. https://getbootstrap.com/docs/5.3/

[5] Python Software Foundation. (2024). *decimal — Decimal fixed point arithmetic*. https://docs.python.org/3/library/decimal.html

[6] Fowler, M. (2003). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

[7] OWASP Foundation. (2023). *OWASP Top Ten Web Application Security Risks*. https://owasp.org/www-project-top-ten/

[8] Gunicorn Team. (2024). *Gunicorn — Python WSGI HTTP Server*. https://gunicorn.org/

[9] Nginx Inc. (2024). *Nginx Documentation*. https://nginx.org/en/docs/

[10] Object Management Group. (2017). *Unified Modeling Language Specification, v2.5.1*. https://www.omg.org/spec/UML/2.5.1/